

# Revision: likelihood and regression

Andrew Parnell

## Learning outcomes

- ▶ Know a few more probability distributions
- ▶ Understand how likelihood works
- ▶ Know how to choose a probability distribution for some data
- ▶ Be able to create and interpret output from a likelihood estimation
- ▶ Understand how to maximise a likelihood for a linear regression

# Likelihood and inference

- ▶ A big chunk of statistics involves fitting probability distributions to data
- ▶ The definition of data can be very broad
- ▶ We have to choose a suitable probability distribution
- ▶ We have to choose a fitting method
- ▶ We obtain parameter estimates of these probability distributions

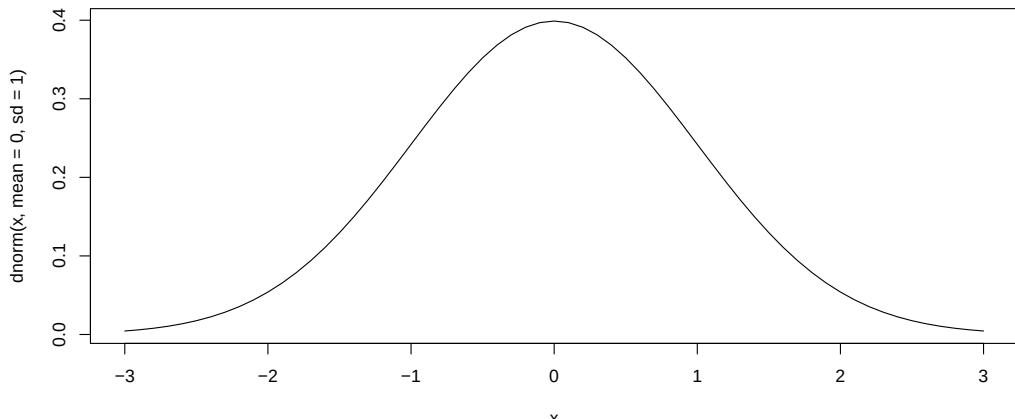
We use these parameter estimates to make numerical predictions about the future

# What is a probability distribution?

- ▶ A probability distribution is just a mathematical tool to predict the behaviour of random data
- ▶ A probability distribution can come in (at least) three different forms:
  1. As a picture, usually represented in a bar chart or a line
  2. As an equation, which enables us to calculate probabilities (or probability densities)
  3. As a set of samples (similar to a data set) which follow the shape of the probability distribution

## Example: the normal distribution - picture

```
x = seq(from = -3, to = 3, by = 0.1)
plot(x, dnorm(x, mean = 0, sd = 1), type = 'l')
```



## Example: the normal distribution - equation

$$f(x; \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- ▶ Here  $\mu$  (the mean) and  $\sigma$  (the standard deviation) are the parameters of the normal distribution
- ▶ If someone gives us some data and says to 'fit' a normal distribution they mean to estimate  $\mu$  and  $\sigma$  for their data
- ▶ This distribution is *continuous* which means that the values of  $x$  can be real numbers
- ▶ The function in R `dnorm` just works out the above formula for given values of  $x$ ,  $\mu$ , and  $\sigma$

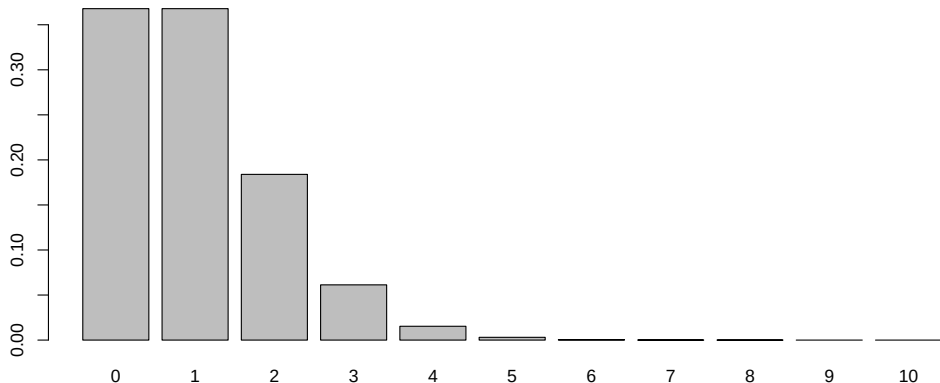
## Example: the normal distribution - samples

```
rnorm(20, mean = 0, sd = 1)
```

```
## [1] 0.383179380 0.769040938  
## [3] 0.347344258 0.896997520  
## [5] 1.484240267 0.639685792  
## [7] 0.419788398 -2.312811717  
## [9] 0.870870379 0.124457770  
## [11] 0.827932510 0.560235813  
## [13] 0.643659986 0.386744347  
## [15] 2.233268189 0.556978513  
## [17] 0.984250489 -0.001812099  
## [19] -0.219153476 -0.050811644
```

## Example 2: the Poisson distribution - picture

```
x = seq(from = 0, to = 10, by = 1)
barplot(dpois(x, lambda = 1), names.arg = x)
```





## Example 2: the Poisson distribution - equation

$$f(x; \lambda) = \frac{e^{-\lambda} \lambda^x}{x!}$$

- ▶ The only parameter in the Poisson distribution is  $\lambda$  which represents both the mean and the variance, which have to be equal
- ▶ The Poisson is a discrete probability distribution, which means that the data values have to be integers
- ▶ Due to the strange mean = variance relationship, the Poisson distribution rarely matches real-world situations

## Example 2: the Poisson distribution - samples

```
rpois(100, lambda = 1)
```

```
##      [1] 1 1 2 1 1 2 4 0 0 1 1 0 1 4 0 0 0  
##     [18] 2 1 0 1 1 1 2 2 2 0 2 0 0 0 0 1 0  
##     [35] 1 2 0 5 2 0 4 1 1 4 0 1 0 2 1 0 2  
##     [52] 1 0 2 0 1 0 1 0 1 2 1 1 1 2 2 2 0  
##     [69] 1 1 0 0 0 0 0 0 0 3 0 0 0 1 1 2 0  
##     [86] 0 3 1 1 0 0 2 2 1 1 0 2 0 0 0
```

## A few more useful probability distributions

- ▶ Binomial. Discrete. Useful when the values have a lower and upper limit. Has two parameters  $N$  and  $p$  which determine how the values behave. Mean is  $Np$  and standard deviation  $\sqrt{Np(1-p)}$ . R function is `dbinom`
- ▶  $t$ -distribution. Continuous. Looks like a normal distribution but has fatter tails. Good for capturing data with outliers. Has three parameters, mean, scale (like standard deviation), and shape to measure the heavy tailed-ness. R function is `dt`
- ▶ Gamma distribution. Continuous but restricted to positive values only. Has two parameters shape and rate, which aren't easily interpretable. R function is `dgamma`
- ▶ Uniform distribution. Continuous but restricted to a lower and upper range. Shape of graph is completely flat. R function is `dunif`

# Choosing a probability distribution

- ▶ There is always a probability distribution for your data
- ▶ When faced with a new set of data, which probability distribution should we choose?
  1. Think about the type of data. Is it discrete or continuous?
  2. Think about the range of values. Are there hard upper and lower limits?
  3. Think about the likely shape of the distribution. Will there be extreme values?
- ▶ Always plot the probability distribution of the data using e.g. `hist`
- ▶ If in doubt, use the normal distribution

# How can we fit a probability distribution to some data?

By fitting a probability distribution to some data we mean *estimating the parameters of that distribution*

We will cover three techniques:

- ▶ The simple way (method of moments)
- ▶ The harder way (likelihood)
- ▶ The even harder way (Bayes; later)

## Method of moments

Suppose we have 10 data points and we want to fit a normal distribution:

```
x = c(-0.23, -0.97, -1.94, -2.08, 2.55,  
      0.8, -0.03, -0.79, -0.6, 1.75)
```

The normal distribution has two parameters representing the mean and the standard deviation, so just calculate these values from the data and set  $\hat{\mu} = \text{mean}(x)$  and  $\hat{\sigma} = \text{sd}(x)$ :

```
mean(x); sd(x)
```

```
## [1] -0.154
```

```
## [1] 1.492077
```

## Notes on the method of moments

- ▶ It's called the method of moments because the mean and the standard deviation are known as the moments of a probability distribution
- ▶ We write hat on the parameters (e.g.  $\hat{\mu}$ ) to make it clear that these are parameter estimates and not the true values
- ▶ The method gets a little bit fiddlier when the parameters don't represent the moments directly. For example with the binomial we'd need to solve a simultaneous equation
- ▶ The method becomes unworkable once we move into situations with more than a few parameters

# Likelihood

- ▶ Main idea: use the probability distribution formula to find how likely each data point is, and then multiply it all together
- ▶ Steps:
  1. Guess some values of the parameters
  2. For each data point, calculate the probability distribution formula
  3. Multiply the lot together to give the likelihood
  4. Repeat from 1 with different values of the parameters

We choose the values of the parameters to maximise the likelihood



# Calculating the likelihood in practice

In R, it's one line!

```
prod(dnorm(x, mean = 0, sd = 1))
```

```
## [1] 4.042974e-09
```

```
prod(dnorm(x, mean = 1, sd = 1))
```

```
## [1] 5.840029e-12
```

- The likelihood when the mean is 0 is higher than the likelihood when it is 1, indicating that these parameter values are better supported by the data

## Logs

- ▶ Likelihood values tend to get very small very quickly, so most people instead work with the log of the likelihood, which is also easier to calculate
- ▶ We usually compute the log of the probability distribution and sum these values (recall the log of the product is the same as the sum of the logs)

```
log(prod(dnorm(x, mean = 0, sd = 1)))
```

```
## [1] -19.32629
```

```
sum(dnorm(x, mean = 0, sd = 1, log = TRUE))
```

```
## [1] -19.32629
```

- ▶ We thus find the parameter values that maximise the log of the likelihood rather than the likelihood itself

# Maximising the likelihood

- ▶ One way to find the 'best' parameters is to try lots and lots of different values and take the ones that provide the biggest log likelihood. This is very inefficient
- ▶ Another way is to use mathematics - we can often maximise the likelihood using calculus. (We are not going to do this)
- ▶ R has a number of very efficient built-in optimisation routines (e.g. `nlminb`) which will find the best values for us

## Maximising the likelihood for linear regression

- ▶ In linear regression we can also maximise the likelihood
- ▶ Suppose we have our data in two variables, called  $y$  (response) and  $x$  (explanatory)
- ▶ We have 3 parameters now: an intercept, a slope, and a residual standard deviation
- ▶ Code to calculate the log likelihood would be:

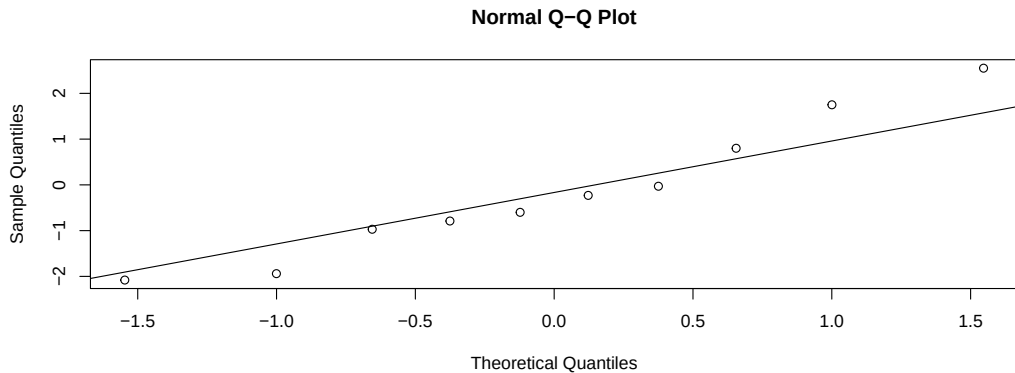
```
sum(dnorm(y, mean = 3 + 2*x, sd = 1, log = TRUE))
```

## Checking the model

- ▶ We could choose almost any probability distribution and still get estimates of the parameters
- ▶ We would also like to know whether the probability distribution we chose actually matches the data
- ▶ One neat way is a QQ-plot (Quantile-Quantile plot) which compares the quantiles of the data with the expected quantiles of the fitted probability distribution
- ▶ If the probability distribution fits the data well the points should lie on a straight line

# A QQ-plot

```
qqnorm(x)  
qqline(x)
```



# How do we estimate the uncertainty in parameters?

- ▶ Having the 'best' values of the parameters isn't the whole story
- ▶ We only have a limited set of data so we can't possibly know the true values exactly. We need to have estimates of uncertainty about the parameters
- ▶ With only a small set of data, the parameter uncertainty should be large
- ▶ With a larger data set, we should be able to get closer to the 'true' values

## Calculating the uncertainty in the parameters

- ▶ There is some mathematical theory which says that the estimated parameters are normally distributed themselves
- ▶ The standard deviation of the estimated parameters depends on the second derivative of the likelihood. Again, we will leave this to the experts
- ▶ R can calculate for us the standard deviation estimates of the parameters
- ▶ This means that we can get an estimated standard deviation of an estimated standard deviation!
- ▶ The estimated standard deviation is usually called the *standard error*
- ▶ For the normal distribution we know that 68% of samples lie approximately within 1 standard deviation, whilst 95% of samples lie within approximately 2 standard deviations



## p-values

- ▶ Some people, rather than compute confidence intervals, calculate  $p$ -values instead
- ▶ These are based on testing a null hypothesis that a particular parameter value is (usually) 0
- ▶ (Can anyone remember the proper definition of a  $p$ -value?)
- ▶ I don't find these very useful but you will see them in lots of the default R output
- ▶ The `lm` function creates these everywhere, but the later packages (`simmr` and `MixSIAR`) do not use them

# Summary

- ▶ Choosing a probability distribution is hard. For most SIMMs the normal distribution is usually the default choice
- ▶ Once you have a chosen model you can compute a likelihood
- ▶ You can use an optimisation method to get you the best parameter values
- ▶ Later in the course we will meet a method (called MCMC) which finds the best values, also gets the uncertainty